

String

What is a String in Java?

- A String is basically a sequence of characters (like text, words, sentences, or even symbols).
- In Java, String is a class in the java.lang package.
- Unlike C/C++, in Java you don't need to worry about '\0' (null character) at the end. Java automatically handles that internally.

Why do we need Strings?

- To **store and manipulate text** like names, emails, messages, sentences, etc.
- Example: "Hello", "Java", "123", "Hi Ajay!"

Ways to Create String Objects

1. Using String Literals

java

 Copy code

```
String str1 = "Hello"; // stored in String Pool
```

 Most common way. Java optimizes memory using something called **String Pool**.

2.Using new Keyword

java

 Copy code

```
String str2 = new String("Hello"); // stored in Heap memory
```

👉 This always creates a new object in heap memory (not reused like String literals).

Syntax Examples

java

 Copy code

```
// String literal
String name = "Ajay";

// Using new keyword
String city = new String("Pune");

// Empty string
String empty = "";

// Concatenation
String greet = "Hello" + " World";
```

Important Concept

String is immutable → once created, you **cannot change** the value of a String object.

Example:

java

 Copy code

```
String s = "Hello";  
s = s + " Java";  
// "Hello" is not changed, a new object "Hello Java" is created.
```

Relation with Character Array

In C/C++, a string is just a **1D array of characters ending with '\0'**.

In Java:

1. Strings are internally character arrays, but we don't have to manually add '\0'.
2. Java gives us the String class with built-in methods like `.length()`, `.toUpperCase()`, `.substring()` etc.

Example:

java

 Copy code

```
char[] letters = {'J','a','v','a'};  
String str = new String(letters);  
System.out.println(str); // Output: Java
```

A String in Java = An object that stores a sequence of characters, immutable, easy to manipulate using built-in methods.

◆ Memory Allocation of Strings in Java

In Java, when we create a String, it can be stored in two main places in memory:

1. String Constant Pool (SCP)

- Part of Heap memory.
- Stores string literals (e.g., "Hello").
- If a string with the same value already exists in the pool, Java reuses that object (saves memory).
- Managed by JVM.

java

 Copy code

```
String s1 = "Java";  
String s2 = "Java"; // Reuses the same object from SCP
```

-  Both s1 and s2 point to the same memory location in SCP.

-  s1 == s2 → true

2. Heap Memory (Outside SCP)

- When you use new String("..."), it always creates a **new object** in heap memory, even if the same literal exists in SCP.
- This gives you two objects:
 - One in SCP (for the literal).
 - One in Heap (created using new).

java

 Copy code

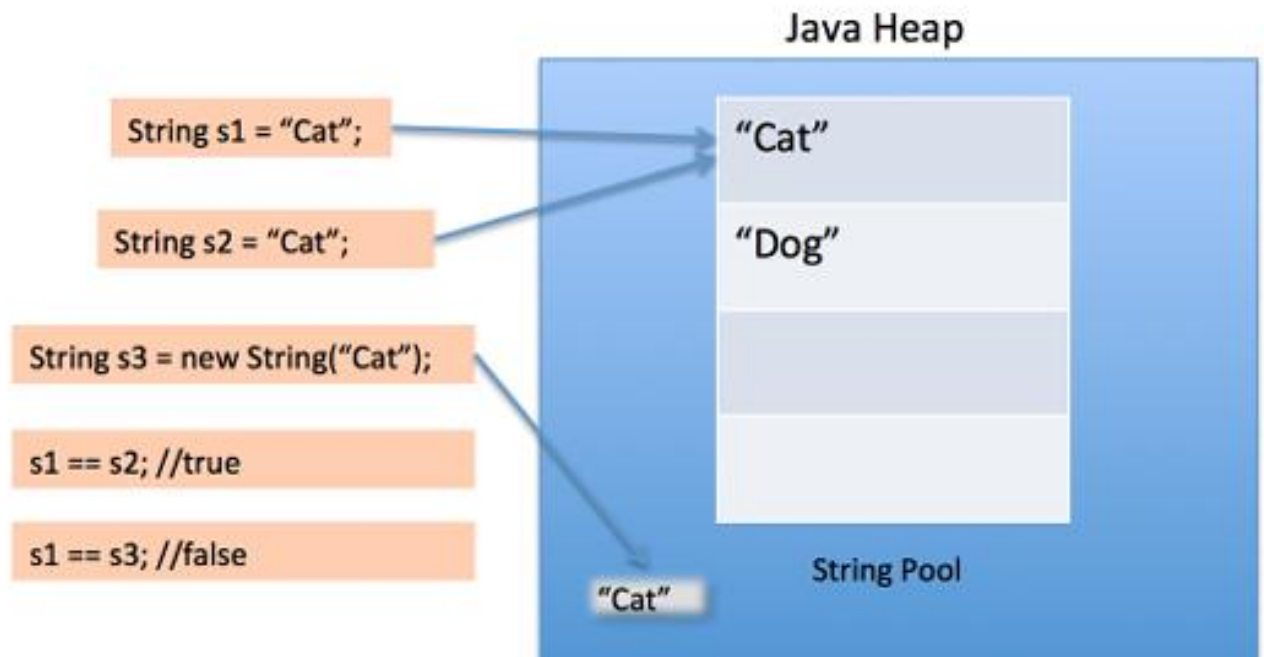
```
String s3 = new String("Java");  
String s4 = new String("Java");
```

-  s3 and s4 are **different objects** in heap.

-  s3 == s4 → false

-  But s3.equals(s4) → true (because values are same).

Visual Representation



vbnet

 Copy code

Heap Memory

```
|— String Constant Pool (SCP)  
|   "Java" ← reused by s1 and s2  
|  
|— Outside SCP  
|   new String("Java") → s3  
|   new String("Java") → s4
```

◆ Example Program

java

 Copy code

```
public class StringMemory {  
    public static void main(String[] args) {  
        String s1 = "Hello";  
        String s2 = "Hello";          // Same object from SCP  
        String s3 = new String("Hello"); // New object in Heap  
  
        System.out.println(s1 == s2); // true (same reference in SCP)  
        System.out.println(s1 == s3); // false (different memory)  
        System.out.println(s1.equals(s3)); // true (same value)  
    }  
}
```

✓ Key Points to Remember

1. String literals → stored in String Constant Pool (SCP).
2. new String() → creates object in Heap, plus literal in SCP (if not already present).
3. Strings are immutable, so every change creates a new object.

Difference Between SCP and Heap Memory

Feature	SCP (String Constant Pool)	Heap Memory (Normal Heap)
Location	Inside Heap, but a special area managed by JVM.	General heap memory allocated to all objects.
Purpose	Stores string literals ("Hello", "Java") for reuse.	Stores objects created using new, including new String("...").
Object Creation	If a string already exists, reuses the same object → avoids duplicates.	Always creates a new object, even if the same value exists elsewhere.
Memory Efficiency	Very memory-efficient because of reusability.	Less efficient since duplicate objects may exist.
Example	<code>String s1 = "Java"; String s2 = "Java";</code> → both point to the same object in SCP.	<code>String s3 = new String("Java"); String s4 = new String("Java");</code> → two different objects in heap.
Comparison (==)	Two literals with the same value → true (same reference).	Two new strings with the same value → false (different references).
Garbage Collection	Objects in SCP are not garbage collected easily (they live till class loader is alive).	Objects in heap are eligible for GC when no reference is left.

Example to Understand

java

 Copy code

```
public class Test {  
    public static void main(String[] args) {  
        String s1 = "Hello";           // SCP  
        String s2 = "Hello";           // Reuses SCP object  
        String s3 = new String("Hello"); // Heap object  
  
        System.out.println(s1 == s2); // true (same SCP object)  
        System.out.println(s1 == s3); // false (different memory location)  
        System.out.println(s1.equals(s3)); // true (values are same)  
    }  
}
```

Quick Summary

- **SCP** → Special part of heap, stores **literals**, avoids duplicates, memory-efficient.
- **Heap (normal)** → Stores objects (including strings created with new), always new allocation.

Commonly Used String Methods in Java

1. length()

👉 Returns the number of characters in the string.

```
java Copy code  
  
public class StringMethods {  
    public static void main(String[] args) {  
        String str = "Hello Java";  
        System.out.println("Length: " + str.length());  
    }  
}
```

✅ Output: Length: 10
(Spaces are also counted.)

2. charAt(int index)

👉 Returns the character at a given index (0-based).

```
java Copy code  
  
String str = "Java";  
System.out.println(str.charAt(0)); // J  
System.out.println(str.charAt(3)); // a
```

3. substring(int begin, int end)

👉 Extracts part of a string.

```
java Copy code  
  
String str = "Programming";  
System.out.println(str.substring(0, 6)); // Progra  
System.out.println(str.substring(3));    // gramming (till end)
```

4. equals(String another) & equalsIgnoreCase(String another)

👉 Compares two strings (case-sensitive & case-insensitive).

```
java Copy code  
  
String s1 = "Hello";  
String s2 = "hello";  
  
System.out.println(s1.equals(s2));           // false  
System.out.println(s1.equalsIgnoreCase(s2)); // true
```

5. toUpperCase() and toLowerCase()

👉 Converts string to uppercase or lowercase.

```
java Copy code  
  
String s = "Java Programming";  
System.out.println(s.toUpperCase()); // JAVA PROGRAMMING  
System.out.println(s.toLowerCase()); // java programming
```

6. trim()

👉 Removes leading and trailing spaces.

java

 Copy code

```
String s = "  Hello World  ";  
System.out.println(s.trim()); // "Hello World"
```

7. replace(char old, char new) / replace(String old, String new)

👉 Replaces characters or substrings.

java

 Copy code

```
String s = "Java is fun";  
System.out.println(s.replace('a','o')); // Jovo is fun  
System.out.println(s.replace("fun","cool")); // Java is cool
```

8. contains(CharSequence seq)

👉 Checks if a string contains a sequence.

java

 Copy code

```
String s = "Learning Java";  
System.out.println(s.contains("Java")); // true  
System.out.println(s.contains("Python")); // false
```

9. indexOf() and lastIndexOf()

👉 Finds the index of a character or substring.

java

 Copy code

```
String s = "Java Programming";  
System.out.println(s.indexOf('a'));    // 1 (first occurrence)  
System.out.println(s.lastIndexOf('a')); // 11 (last occurrence)
```

10. split(String regex)

👉 Splits string into an array based on a delimiter.

java

 Copy code

```
String s = "apple,banana,grapes";  
String[] fruits = s.split(",");  
  
for(String f : fruits) {  
    System.out.println(f);  
}
```

✅ **Output:**

nginx

 Copy code

apple

banana

grapes

11. startsWith() and endsWith()

👉 Checks prefix or suffix.

java

 Copy code

```
String s = "Java Programming";  
System.out.println(s.startsWith("Java")); // true  
System.out.println(s.endsWith("ing"));    // true
```

12. isEmpty() / isBlank() (Java 11+)

👉 Checks if string is empty or blank.

java

 Copy code

```
String s1 = "";  
String s2 = "  ";  
  
System.out.println(s1.isEmpty()); // true  
System.out.println(s2.isBlank()); // true (only spaces)
```

◆ Quick Summary

- **length()** → count characters
- **charAt()** → get character by index
- **substring()** → extract part of string
- **equals()** / **equalsIgnoreCase()** → compare strings
- **toUpperCase()**, **toLowerCase()** → case conversion
- **trim()** → remove spaces
- **replace()** → replace text
- **contains()** → check substring
- **indexOf()**, **lastIndexOf()** → find positions
- **split()** → break string into array
- **startsWith()**, **endsWith()** → prefix/suffix check
- **isEmpty()**, **isBlank()** → check empty/blank

```

public class stringdemo {
    public static void main(String[] args) {

        String name="fortunecloud";

        System.out.println("String is:"+name);
        System.out.println("length of the string is :"+name.length());
        System.out.println("uppercase:"+name.toUpperCase());
        System.out.println("lowercase:"+name.toLowerCase());
        System.out.println("equals:"+name.equals("fortunEcloud"));
        System.out.println("char at:"+name.charAt(5));
        System.out.println("indexOf:"+name.indexOf("o"));
        System.out.println("ends with:"+name.endsWith("d"));
        System.out.println("starts with:"+name.startsWith("h"));
        System.out.println("replace:"+name.replace("e", "a"));
        System.out.println("concat:"+name.concat("technology"));
        System.out.println("substring:"+name.substring(7));
        System.out.println("substring with begain and end:"+name.substring(3,7));
        System.out.println("trim :"+name.trim());

        char arr[]=name.toCharArray();
        System.out.println(Arrays.toString(arr));

        String sarray[]=name.split("/s");
        System.out.println(Arrays.toString(sarray));

        System.out.println("replace all:"+name.replaceAll("/s"," "));
    }
}

```

◆ Code Walkthrough

```
String name = "fortunecloud";
```

👉 A String object is created in **String Constant Pool (SCP)** with value "fortunecloud".

✅ String Methods Used

```
System.out.println("String is:" + name);
```

Output: String is:fortunecloud

java

 Copy code

```
System.out.println("length of the string is :" + name.length());
```

👉 length() → returns total number of characters.

Output: length of the string is :12

java

 Copy code

```
System.out.println("uppercase:" + name.toUpperCase());  
System.out.println("lowercase:" + name.toLowerCase());
```

👉 Converts to **UPPERCASE** and **lowercase**.

Output:

makefile

 Copy code

```
uppercase:FORTUNECLLOUD  
lowercase:fortunecLOUD
```

java

 Copy code

```
System.out.println("equals:" + name.equals("fortunEcloud"));
```

👉 equals() → checks if strings are equal (case-sensitive).

Output: false (because E ≠ e)

java

 Copy code

```
System.out.println("char at:" + name.charAt(5));
```

👉 charAt(5) → 0-based index.

fortunecloud → f(0), o(1), r(2), t(3), u(4), **n(5)**

Output: n

java

 Copy code

```
System.out.println("indexof:" + name.indexOf("o"));
```

👉 Finds the first index of "o".

fortunecloud → "o" first appears at index 1.

Output: 1

java

 Copy code

```
System.out.println("ends with:" + name.endsWith("d"));
System.out.println("starts with:" + name.startsWith("h"));
```

👉 Checks prefix/suffix.

- Ends with "d" →  true
- Starts with "h" →  false (it starts with "f")

Output:

csharp

 Copy code

```
ends with:true  
starts with:false
```

java

 Copy code

```
System.out.println("replace:" + name.replace("e", "a"));
```

👉 Replaces **all e** → **a**.

fortunecloud → fortunacloud

Output: fortunacloud

java

 Copy code

```
System.out.println("comcat:" + name.concat("technology"));
```

👉 Concatenates "technology".

Output: fortunecloudtechnology

java

 Copy code

```
System.out.println("substring:" + name.substring(7));  
System.out.println("substring with begain and end:" + name.substring(3,7));
```

👉 Extracts substring.

- `substring(7)` → "loud"
- `substring(3,7)` → "tune" (from index 3 to 6)

Output:

vbnet

 Copy code

```
substring:loud  
substring with begain and end:tune
```

java

 Copy code

```
System.out.println("trim :" + name.trim());
```

👉 Removes spaces (but here no spaces, so same string).

Output: fortunecloud

java

 Copy code

```
char arr[] = name.toCharArray();  
System.out.println(Arrays.toString(arr));
```

👉 Converts to char array.

Output: [f, o, r, t, u, n, e, c, l, o, u, d]

java

 Copy code

```
String sarray[] = name.split("/s");  
System.out.println(Arrays.toString(sarray));  
  
System.out.println("replace all:" + name.replaceAll("/s", " "));
```

⚠ **Issue here** → In Java, regex for whitespace is `\\s` (not `/s`).

So:

- `split("/s")` will not split correctly → it will return the full string as 1 element.
- `replaceAll("/s", " ")` will not work as expected.



Correct code:

java

 Copy code

```
String sarray[] = name.split("\\s");  
System.out.println(Arrays.toString(sarray));  
  
System.out.println("replace all:" + name.replaceAll("\\s", " "));
```

Fixed / Corrected Output for Last Part

Since "fortunecloud" has **no spaces**, output will still be the same string:

CSS

 Copy code

```
[fortunecloud]  
replace all:fortunecloud
```

◆ String Methods in Java – Summary

1 Basic Info

- `length()` → returns length of string.
 - `charAt(int index)` → returns character at given index.
 - `toCharArray()` → converts string → char array.
-

2 Comparison

- `equals(str)` → compares values (case-sensitive).
 - `equalsIgnoreCase(str)` → compares values ignoring case.
 - `compareTo(str)` → compares lexicographically (returns +, -, 0).
 - `compareToIgnoreCase(str)` → compares ignoring case.
-

3 Substrings

- `substring(int begin)` → returns substring from begin to end.
- `substring(int begin, int end)` → substring between indices.

4 Searching

- `indexOf(char/str)` → first occurrence index.
 - `lastIndexOf(char/str)` → last occurrence index.
 - `contains(str)` → checks if substring exists.
 - `startsWith(str)` → true if string starts with given prefix.
 - `endsWith(str)` → true if string ends with given suffix.
-

5 Modification

- `concat(str)` → joins two strings.
 - `replace(old, new)` → replaces characters/substring.
 - `replaceAll(regex, new)` → replaces using regex.
 - `trim()` → removes leading & trailing spaces.
-

6 Case Conversion

- `toUpperCase()` → converts to UPPERCASE.
 - `toLowerCase()` → converts to lowercase.
-

7 Splitting & Joining

- `split(regex)` → splits into string array by regex.
- `join(delimiter, elements...)` → joins multiple strings with delimiter (Java 8+).

8 Checking

- isEmpty() → true if length = 0.
- isBlank() → true if only spaces or empty (Java 11+).

9 Utility

- valueOf() → converts primitive → string.
- format() → formatted string like printf.
- intern() → forces string into SCP (String Constant Pool).

1. String

👉 A String is a **sequence of characters**.

- **Immutable** → Once created, cannot be changed.
- Stored in **String Constant Pool (SCP)** if created using literals.
- Any modification (like concatenation) creates a **new object**

✓ Example

java

Copy code

```
public class StringExample {  
    public static void main(String[] args) {  
        String s1 = "Hello";  
        s1 = s1 + " World"; // creates new object  
        System.out.println(s1);  
    }  
}
```

nginx

Copy code

```
Hello World
```

Explanation: "Hello" is not changed; instead "Hello World" is a new object.

◆ 2. StringBuffer

👉 A StringBuffer is a **mutable sequence of characters**.

- **Mutable** → we can modify the same object (append, insert, delete).
- **Thread-safe** → all methods are **synchronized** (safe in multithreaded environment).
- Slower compared to StringBuilder because of synchronization overhead.

✅ Example

java

Copy code

```
public class StringBufferExample {  
    public static void main(String[] args) {  
        StringBuffer sb = new StringBuffer("Hello");  
        sb.append(" World"); // modifies the same object  
        sb.insert(5, " Java"); // insert at index 5  
        sb.replace(0, 5, "Hi"); // replace "Hello" with "Hi"  
        sb.delete(2, 5); // delete chars from index 2 to 4  
        sb.reverse(); // reverse string  
  
        System.out.println(sb);  
    }  
}
```

Output: (step by step transformations)

nginx

Copy code

```
Hello → Hello World → Hello Java World → Hi Java World → HiWorld → dlrow iH
```

3. StringBuilder

👉 A StringBuilder is also a **mutable sequence of characters**.

- Same as StringBuffer but **not synchronized** → **faster** in single-threaded applications.
- Introduced in Java 5

✅ Example

```
java 📄 Copy code  
  
public class StringBuilderExample {  
    public static void main(String[] args) {  
        StringBuilder sb = new StringBuilder("Fortune");  
        sb.append(" Cloud");  
        sb.insert(7, " Technology");  
        sb.delete(0, 7);  
        sb.reverse();  
  
        System.out.println(sb);  
    }  
}
```

Output:

```
nginx 📄 Copy code  
  
duo!C ygo!onhceT
```

Key Differences

Feature	String (Immutable)	StringBuffer (Mutable, Thread-safe)	StringBuilder (Mutable, Faster)
Mutability	Immutable	Mutable	Mutable
Thread Safety	Not thread-safe	Thread-safe (synchronized)	Not thread-safe
Performance	Slow (creates new object)	Slower than StringBuilder	Faster
Use Case	When text is rarely modified (e.g., constants, configs)	When text is modified often in multi-threaded apps	When text is modified often in single-threaded apps

✅ Summary in Simple Words:

- String = Fixed, immutable (new object every time you change).
- String Buffer = Flexible, mutable, but slower (thread-safe).
- StringBuilder = Flexible, mutable, and fastest (not thread-safe).

◆ Example Program with All Three

```
java Copy code  
  
public class StringDemo {  
    public static void main(String[] args) {  
        // String  
        String s = "Hello";  
        s = s + " World";  
        System.out.println("String: " + s);  
  
        // StringBuffer  
        StringBuffer sb = new StringBuffer("Hello");  
        sb.append(" World");  
        System.out.println("StringBuffer: " + sb);  
  
        // StringBuilder  
        StringBuilder sbd = new StringBuilder("Hello");  
        sbd.append(" World");  
        System.out.println("StringBuilder: " + sbd);  
    }  
}
```

Output

```
makefile Copy code  
  
String: Hello World  
StringBuffer: Hello World  
StringBuilder: Hello World
```

String Task:-

Without using charAt()

```
import java.util.Scanner;

public class withoutusingcharAt {

    public static void main(String[] args) {

        //take string input from user
        System.out.println("enter the string:");
        Scanner sc=new Scanner(System.in);
        String s=sc.next(); // user enters string

        //take index
        System.out.println("enter the index");
        int index=sc.nextInt(); // user enters index (e.g., 5)

        //convert string into character array
        char chr[]=s.toCharArray(); // example: "cravita" → ['c','r','a','v','i','t','a']

        //loop through character array
        for(int i=0;i<chr.length;i++) {
            if(index==i) {
                System.out.println(chr[i]);
            }
        }
    }
}
```



Explanation:

1. Import Scanner

- You use Scanner class to take input from the user.

2. Input String from User

```
java

String s=sc.next();
```

 Copy code

Suppose user enters:

nginx

 Copy code

cravita

3.Input Index from User

java

 Copy code

```
int index=sc.nextInt();
```

- Suppose user enters:

 Copy code

5

4.Convert String to Character Array

java

 Copy code

```
char chr[]=s.toCharArray();
```

- "cravita" → ['c','r','a','v','i','t','a']
- Index positions:

css

 Copy code

0 → c

1 → r

2 → a

3 → v

4 → i

5 → t

6 → a

5.Loop through Array

java

 Copy code

```
for(int i=0;i<chr.length;i++) {  
    if(index==i) {  
        System.out.println(chr[i]);  
    }  
}
```

- The loop checks each index i with the given index.
- When i == 5, condition is true, so chr[5] (which is 't') is printed.



Example Execution

Input:

yaml

 Copy code

```
enter the string: cravita  
enter the index: 5
```

Output:

nginx

 Copy code

t



Key Points

- Instead of using **charAt(index)**, you used **toCharArray()** and accessed elements by index.
- This is a **manual way** of getting a character at a specific position.
- Works the same as **s.charAt(index)** but uses arrays.

Task 2:

```
java Copy code  
  
package assignment;  
  
public class withusingindex {  
  
    public static void main(String[] args) {  
  
        String s="Himanshu"; // original string  
        char ch='s';          // character we want to find  
  
        char arr[]=s.toCharArray(); // convert string into character array  
  
        for(int i=0; i<arr.length;i++) { // loop through array  
            if(arr[i]==ch) { // check if current char == 's'  
                System.out.println(i); // print index position  
            }  
        }  
    }  
}
```



Explanation

1. String Creation

```
java Copy code  
  
String s="Himanshu";
```

- You have a string "Himanshu".

2.Character to Find

java

 Copy code

```
char ch='s';
```

- You want to find the index of the character 's' in "Himanshu".

3.Convert String → Character Array

java

 Copy code

```
char arr[]=s.toCharArray();
```

- "Himanshu" becomes:

 Copy code

```
arr[0] = H  
arr[1] = i  
arr[2] = m  
arr[3] = a  
arr[4] = n  
arr[5] = s  
arr[6] = h  
arr[7] = u
```

4. Loop Through Array

java

 Copy code

```
for(int i=0; i<arr.length;i++) {
```

- Iterates from i=0 to i=7.

5. Check Each Character

java

 Copy code

```
if(arr[i]==ch) {  
    System.out.println(i);  
}
```

- Compares each element with 's'.
- When i=5, arr[5] = s, so condition is **true**.
- Prints 5.



Output

 Copy code

5



Key Points

- You are **finding the index of a given character** in a string without using built-in methods like `indexOf()`.
- Works fine even if the character appears multiple times (it will print all positions).
 - Example: "Mississippi" with `ch='s'` → prints 2, 3, 5, 6.

Part 1: Explanation of Your StringBuilder Code

```
java Copy code  
  
public class StringBuilderdemo {  
    public static void main(String[] args) {  
  
        System.out.println("_____StringBuffer_____");  
        StringBuilder str1=new StringBuilder ("good");  
  
        // Original string  
        System.out.println("before append:"+str1);    // good  
  
        // Append text  
        System.out.println("using append:" +str1.append("students")); // goodstud  
        System.out.println("after append"+str1);      // goodstudents  
  
        // Reverse the string  
        System.out.println("after reverse:"+str1.reverse()); // stnedutsdoog  
  
        // Find index of 'o'  
        System.out.println("index of :"+str1.indexOf("o")); // 8  
  
        // Character at index 0  
        System.out.println("charat:"+str1.charAt(0)); // s
```

 Copy code

```
System.out.println("_____StringBuffer_____");
StringBuilder str1=new StringBuilder ("good");

// Original string
System.out.println("before append:"+str1);    // good

// Append text
System.out.println("using append:" +str1.append("students")); // goodstudents
System.out.println("after append"+str1);      // goodstudents

// Reverse the string
System.out.println("after reverse:"+str1.reverse()); // stnedutsdoog

// Find index of 'o'
System.out.println("index of :"+str1.indexOf("o")); // 8

// Character at index 0
System.out.println("charat:"+str1.charAt(0)); // s

// Index of substring "t" starting from index 3
System.out.println("index of substring:"+str1.indexOf("t",3)); // 3
```

```
// Reverse the string
System.out.println("after reverse:"+str1.reverse()); // stnedutsdoog

// Find index of 'o'
System.out.println("index of :"+str1.indexOf("o")); // 8

// Character at index 0
System.out.println("charat:"+str1.charAt(0)); // s

// Index of substring "t" starting from index 3
System.out.println("index of substring:"+str1.indexOf("t",3)); // 3

// Insert "hina" at index 10
System.out.println(str1.insert(10, "hina"));

// Delete characters from index 0 to 8
str1.delete(0, 8);
System.out.println("string is:"+str1);

// Repeat is not available directly in StringBuilder before Java 11,
// but assume you have custom repeat
System.out.println(str1.repeat(str1,8));
```

What Each Line Does

1.Create StringBuilder

```
java
StringBuilder str1=new StringBuilder("good");
```

- Creates a mutable string "good".

2. Append

java

 Copy code

```
str1.append("students");
```

- Adds "students" → "goodstudents".

3. Reverse

java

 Copy code

```
str1.reverse();
```

- Reverses string → "stnedutsdoog".

4. indexOf("o")

- Finds position of first 'o' in "stnedutsdoog".
- Result → 8.

5. charAt(0)

- First character of "stnedutsdoog" → 's'.

4. indexOf("t",3)

- Searches for 't' starting at index 3.
- Found at index 3.

5. `insert(10,"hina")`

- Inserts "hina" at index 10.
- "stne hina dutsdoog" (depends on state after reverse).

6. `delete(0,8)`

- Removes characters from index 0 to 7.

7. `repeat(...)`

- ⚠️ `StringBuilder` doesn't have `repeat()` in Java.
- `String.repeat(int count)` exists in Java 11+ but **only for `String`**, not `StringBuilder`.

If you want repeat, you first need to convert to `String`:

java

 Copy code

```
System.out.println(str1.toString().repeat(8));
```

Part 2: String Builder vs String Buffer

✓ Common Features

Both StringBuilder and StringBuffer:

- Are mutable (can be changed after creation).
- Provide methods like append(), insert(), delete(), reverse(), charAt(), indexOf() etc.
- Store sequence of characters.

Feature	StringBuilder	StringBuffer
Thread-safety	✗ Not synchronized (faster, but not thread-safe)	✓ Synchronized (thread-safe, slower)
Performance	Faster (no extra synchronization overhead)	Slower (due to synchronization)
Use case	Single-threaded programs	Multi-threaded programs

Example with StringBuffer (same logic)

```
public class StringBufferDemo {
    public static void main(String[] args) {
        StringBuffer str2 = new StringBuffer("good");

        System.out.println("before append:"+str2);    // good
        System.out.println("using append:" +str2.append("students")); // goodstudents
        System.out.println("after append"+str2);      // goodstudents
        System.out.println("after reverse:"+str2.reverse()); // stnedutsdoog
        System.out.println("index of :"+str2.indexOf("o")); // 8
        System.out.println("charat:"+str2.charAt(0)); // s
        System.out.println("index of substring:"+str2.indexOf("t",3)); // 3
        System.out.println(str2.insert(10, "hina"));
        str2.delete(0, 8);
        System.out.println("string is:"+str2);

        // repeat works only on String, not StringBuffer
        System.out.println(str2.toString().repeat(5));
    }
}
```

Final Summary

- StringBuilder: Faster, not synchronized → use in single-threaded apps.
- StringBuffer: Slower, synchronized → use in multi-threaded apps.
- Both provide methods for append, insert, delete, reverse, charAt, indexOf.
- repeat() is only available in String (Java 11+), not in StringBuilder or StringBuffer

Feature	String	StringBuffer	StringBuilder
Nature	Immutable (cannot be changed after creation)	Mutable (can be changed)	Mutable (can be changed)
Thread-Safety	Not thread-safe (but immutable so safe)	✅ Thread-safe (synchronized)	❌ Not thread-safe (not synchronized)
Performance	Slow (creates new object on modification)	Slower (due to synchronization)	Faster (no synchronization)
Use Case	When data is fixed or rarely changed	When data is frequently changed in multi-threaded apps	When data is frequently changed in single-threaded apps

✓ Example in Short

java

 Copy code

```
// String (Immutable)
String s = "Hello";
s.concat(" World");
System.out.println(s); // Output: Hello (unchanged)

// StringBuffer (Mutable + Thread-safe)
StringBuffer sb = new StringBuffer("Hello");
sb.append(" World");
System.out.println(sb); // Output: Hello World

// StringBuilder (Mutable + Faster)
StringBuilder sb2 = new StringBuilder("Hello");
sb2.append(" World");
System.out.println(sb2); // Output: Hello World
```

- String → Permanent, every change makes a new object.
- StringBuffer → Changeable, safe for multi-threading, but slower.
- StringBuilder → Changeable, fast, best for single-threading.

◆ Heap vs SCP (String Constant Pool)

1. Heap Memory

- Big memory area inside JVM.
- Stores **all objects** (Strings, arrays, user-defined objects, etc.).
- Strings created with `new` go here.
- Multiple copies are allowed.
- 🙌 Example:

java

📄 Copy code

```
String s1 = new String("hello");  
String s2 = new String("hello");
```

✅ Here **two objects** will be created in Heap.

2. SCP (String Constant Pool)

- A **small special part** inside Heap memory.
- Stores **only String literals**.
- Avoids duplicate strings → saves memory.
- If the same literal is created, JVM **reuses** the existing one.

🙌 Example:

java

📄 Copy code

```
String s1 = "hello";  
String s2 = "hello";
```

✅ Here only **one object** is created in SCP, and both variables point to the same "hello".

Feature	Heap	SCP
What it stores	All objects (Strings, arrays, user objects)	Only String literals
Duplicates	Allowed	Not allowed (reuses same string)
Memory size	Large	Small (part of Heap)
When created	Using new keyword	Using string literals ("hello")

🎯 Super Easy Way to Remember

- **Heap** → Big house 🏠 where everyone (all objects) can live, even duplicates.
- **SCP** → Special locker 🗝 inside the house, only **unique string literals** are kept, no duplicates.

◆ What is a Method in Java?

- A **block of code** that performs a specific task.
- Helps to **reuse code** and **make programs modular**.
- Similar to a **function** in other languages.

◆ Syntax of a Method

java

 Copy code

```
returnType methodName(parameters) {  
    // method body  
    return value; // if returnType is not void  
}
```

 Example:

java

 Copy code

```
int add(int a, int b) {  
    return a + b;  
}
```

◆ Types of Methods in Java

1. Predefined Methods (Built-in)

- Already provided by Java libraries.
- Example:

java

 Copy code

```
String s = "Java";  
System.out.println(s.length()); // length() is predefined method  
System.out.println(Math.sqrt(25)); // sqrt() from Math class
```

2. User-defined Methods

- Created by programmers for custom logic.
- Example:

```
java Copy code  
  
public class Demo {  
    int square(int x) {  
        return x * x;  
    }  
  
    public static void main(String[] args) {  
        Demo d = new Demo();  
        System.out.println(d.square(5)); // 25  
    }  
}
```

3. Static Methods

- Declared with static keyword.
- Can be called **without creating object**.
- Example:

```
java Copy code  
  
public class Test {  
    static void greet() {  
        System.out.println("Hello, World!");  
    }  
    public static void main(String[] args) {  
        greet(); // directly call  
    }  
}
```

4. Instance Methods (non static)

- Belong to objects.
- Need object creation to call.
- Example:

java

 Copy code

```
public class Car {  
    void run() {  
        System.out.println("Car is running");  
    }  
    public static void main(String[] args) {  
        Car c = new Car();  
        c.run(); // called using object  
    }  
}
```

5. Abstract Methods

- Declared without body (only signature).
- Used in **abstract classes / interfaces**.
- Example:

java

 Copy code

```
abstract class Animal {  
    abstract void sound();  
}  
class Dog extends Animal {  
    void sound() {  
        System.out.println("Bark");  
    }  
}
```

6. Final Methods

- Declared with final keyword.
- Cannot be **overridden** in subclasses.
- Example:

```
java Copy code  
  
class Parent {  
    final void display() {  
        System.out.println("Hello");  
    }  
}  
class Child extends Parent {  
    // cannot override display()  
}
```

7. Synchronized Methods

- Used in **multithreading** to avoid conflicts.
- Only one thread can access at a time.
- Example:

```
java Copy code  
  
class Counter {  
    private int count = 0;  
    synchronized void increment() {  
        count++;  
    }  
}
```

Summary (Simple)

- **Predefined** → Already in Java (length(), sqrt()).
- **User-defined** → Created by us.
- **Static** → No object needed.
- **Instance** → Needs object.
- **Abstract** → Only declaration, no body.
- **Final** → Cannot be overridden.
- **Synchronized** → Thread-safe.

Pratiksha mam –

◆ 1. Static Method

- Declared using static keyword.
- Can be called without creating object.
- Belongs to the class.

 **Example:**

java

 Copy code

```
public class Demo {  
    static void greet() { // static method  
        System.out.println("Hello, World!");  
    }  
    public static void main(String[] args) {  
        greet(); // call directly (no object needed)  
    }  
}
```

 **Output: Hello, World!**

◆ 2. Non-Static Method (Instance Method)

- Declared without static.
- Needs object to be called.
- Belongs to the object.

👉 Example:

java

📄 Copy code

```
public class Car {  
    void run() { // non-static method  
        System.out.println("Car is running");  
    }  
    public static void main(String[] args) {  
        Car c = new Car(); // create object  
        c.run();           // call using object  
    }  
}
```

✅ Output: Car is running

◆ 3. Parameterized Method

- Method with parameters/arguments.
- Used when input values are required.

👉 Example:

```
public class MathOperation {  
    void add(int a, int b) {    // parameters  
        System.out.println("Sum = " + (a + b));  
    }  
    public static void main(String[] args) {  
        MathOperation m = new MathOperation();  
        m.add(10, 20);    // passing values  
    }  
}
```

✅ Output: Sum = 30

◆ 4. Return Type Method

- Method that returns a value using return.
- Return type can be int, String, double, etc.

👉 Example:

```
java 📄 Copy code  
  
public class Square {  
    int findSquare(int x) {    // return type int  
        return x * x;  
    }  
    public static void main(String[] args) {  
        Square s = new Square();  
        int result = s.findSquare(5);  
        System.out.println("Square = " + result);  
    }  
}
```

✅ Output: Square = 25

Method Type	Keyword	Object Needed?	Returns Value?	Example
Static	static	 No	Optional	Math.sqrt(25)
Non-Static	none	 Yes	Optional	obj.run()
Parameterized	any	 / 	Optional	add(int a, int b)
Return Type	return	 / 	 Yes	int square()